

论文解构报告

FlexPipe: Adapting Dynamic LLM Serving Through Inflight Pipeline Refactoring in Fragmented Serverless Clusters

Yanying Lin, Shijie Peng, Chengzhi Lu, Chengzhong Xu, Kejiang Ye

2026

大语言模型

无服务器计算

流水线并行

动态扩缩容

源文件：<https://doi.org/10.1145/3767295.3769316>

DOI：10.1145/3767295.3769316

生成时间：2026-07-20

目录

- 摘要翻译 3
 - 1. 一句话总览 3
 - 2. 阅读范围与证据状态 3
 - 3. 根问题：论文究竟要解决什么 3
 - 4. 旧方法为何不够 4
 - 5. 核心洞见与假设 5
 - 6. 方法：从洞见到可执行系统 5
 - 7. 为什么它可能有效 6
 - 8. 实验与指标：证据是否支撑主张 6
 - 9. 图表解读与论证关系 9
 - 10. 完整因果推理树 14
 - 11. 结论、贡献与边界 15
 - 12. 面向初学者的学习路径 16

在生产环境中部署大语言模型（LLM）服务面临着来自无服务器集群中高度多变的请求模式和严重资源碎片化的重大挑战。当前系统依赖于静态流水线配置，难以适应动态的工作负载条件，从而导致显著的效率低下。我们提出了 FLEXPIPE，这是一种在运行期间动态重新配置流水线架构的新型系统，旨在解决这些根本性限制。FLEXPIPE 将模型分解为细粒度阶段，并根据实时请求模式分析智能调整流水线粒度，实现了三项关键创新：保留计算图约束的细粒度模型分区、具有一致缓存转换的在线（inflight）流水线重构，以及解决 GPU 碎片的拓扑感知资源分配。在包含 82 个 GPU 的集群上进行的全面评估表明，与最先进的系统相比，FLEXPIPE 实现了高达 8.5 倍的资源效率，同时延迟降低了 38.3%，将 GPU 预留需求从峰值容量的 75% 降低到了 30%。

1. 一句话总览

论文元数据

字段	内容
标题	FLEXPIPE
作者	文中未说明
发表场所 / 年份	文中未说明
研究领域	无服务器计算 / 大语言模型推理
关键词	文中未说明

资源链接

- 原始文件：本地上传文件
- arXiv：文中未说明
- DOI：文中未说明
- 代码 / 数据集：文中未说明

标签（3-5 个）：FLEXPIPE · Serverless · LLM 推理 · 流水线重构

[作者明确陈述] FLEXPIPE 是一个面向无服务器（Serverless，指按实际计算量付费且免去人工服务器运维的云计算模式）GPU 集群的大语言模型（LLM，指通过海量文本训练的具有强大语言理解与生成能力的深度学习模型）推理服务系统，它通过在运行时执行无需服务中断的在线流水线重构（Inflight Pipeline Refactoring，指像行驶中的客车不停车调整车厢布局一样，在推理服务运行中动态调整执行拓扑），自适应改变流水线并行粒度，以应对高波动性的工作负载并最大化资源利用率 [页码：1, 2]。

2. 阅读范围与证据状态

[背景知识] 本报告基于对论文全部正文内容（覆盖页码：1-14 页）的解构。目前系统存在 7 张图片超过 max_figure_images 而未进行视觉分析的限制，导致对应的部分实验结果图表设计无法从当前 OCR 视觉内容中确认；另外，论文中未说明开源代码库链接、数据集具体划分细节，且公式 4、公式 10、公式 11 和公式 13 等算法的关键超参数未披露整定数值，这些都构成了当前证据状态的学术缺口。

3. 根问题：论文究竟要解决什么

[作者明确陈述] 在无服务器云环境中部署大语言模型推理服务时，面临极端的工作负载波动性（Workload Volatility，指系统面临的请求流量随时间剧烈变化的特征）与严重的物理 GPU 资源碎片化（Resource Fragmentation，指空闲资源分散在不同物理节点上，缺乏连续大块空闲资源）之间的根本冲突 [页码：1]。

[背景知识] 无服务器架构通过自动弹性伸缩和按需计费能极大降低大模型推理服务（Serving）的运维门槛。然而，大模型极高的参数规模对多 GPU 协同提出了严苛要求，在云环境中难以提供持续的高带宽物理连接 [页码：3]。

[基于文中逻辑的推断] 该冲突的解决难点主要体现在三点：

1. **张量并行在碎片化环境下的失效**：调度程序通常实施反亲和性（Anti-affinity，指为防止单点故障将副本分散部署到不同物理机器上的安全策略）调度，导致可用的 GPU 散落在不同的物理节点上 [页码：3]。张量并行（Tensor Parallelism，指将单层矩阵乘法拆分给多个 GPU 同时计算，像多人共同拼一张拼图）依赖极高带宽的节点内互连（如 NVLink）。而在碎片化物理节点间，由于带宽受限且 All-reduce 同步通信开销巨大，张量并行被迫降级 [页码：3]。生产环境实测表明，在同一台服务器上成功共同定位 4 块 GPU 的概率仅为 0.02%，导致 78% 的张量并行请求被迫退化为流水线并行（Pipeline Parallelism，指让 GPU 像流水线工人一样依次处理模型的部分层） [页码：2, 4] [实验支持]。

2. **静态流水线与波动负载的失配**：真实生产环境的流量变异系数（CV，衡量流量波动剧烈程度的指标，值越大说明流量越忽大忽小）在不同时间窗口下波动可达 7 倍 [页码：1, 2] [实验支持]。采用静态配置的传统流水线无法自适应调整执行拓扑，在流量峰值时易引发排队拥堵和严重的延迟超标，违背服务等级协定（SLO，指服务商对客户承诺的响应时限）[页码：2, 4]。
3. **利用率与保留成本的折衷**：为了应对突发流量，静态流水线系统不得不将高达 75% 历史峰值的 GPU 容量维持为长开（always-on）保留状态，导致日常的 GPU 平均利用率仅为 17%，造成高昂的硬件成本浪费 [页码：1, 2] [实验支持]。
- [基于文中逻辑的推断] 该问题的成功标准是在完全保障服务质量（SLO）的前提下，实现自适应流量波动的动态资源调度，大幅降低常驻 GPU 预留配额，提高资源效率，并缩短延迟和挂起恢复时间 [页码：2, 13]。
- 本文的 story**：[作者明确陈述] 针对 Serverless 流量高波动性与 GPU 物理零碎分布的冲突，传统静态张量并行或流水线部署要么因跨节点网络带宽受限而退化，要么因无法调整级数而在高波动流量下导致空泡比率激增 22 倍并触发排队雪崩 [页码：2, 3, 5]。本文的核心转折在于：系统应在运行时不中断推理服务的前提下，动态改变流水线级数——在高流量（高 CV）时采用深流水线（细粒度）以支持大批处理容量并吸纳排队，在低流量时采用浅流水线（粗粒度）以最小化级间累计通信开销 [页码：5, 6]。FLEXPIPE 通过设计算子级切分、在线重构和自适应弹性伸缩，实现了无需服务中断的平滑执行拓扑切换，成功打破了传统静态设计的局限 [页码：2, 7, 8]。

4. 旧方法为何不够

1. AlphaServe

- 它原本解决什么：[背景知识] 解决模型并行/流水线并行在深度学习 Serving 中的部署问题，通过历史请求痕迹离线优化流水线结构，最大化长期性能 [页码：2, 11, 14]。
- 它依赖的前提或遗留限制：[基于文中逻辑的推断] 依赖历史请求痕迹进行离线参数分区，假定流量模式长期稳定且可预测 [页码：2]。
- 本文批判的具体 limit：[作者明确陈述] 缺乏短期动态适应力，一旦部署便无法在运行时动态重组，在应对高度多变且不可预测的 Serverless 流量时，会产生严重的排队和瓶颈 [页码：2]。
- 本文用哪个模块回应：[基于文中逻辑的推断] **在线流水线重构（Inflight Pipeline Refactoring）** 模块，根据实时监控的流量 CV 和队列长度，在线动态调整流水线阶段粒度 [页码：2, 8]。
- 哪个比较实验检验这一点：[实验支持] **端到端延迟与吞吐量分解（CV=2, 4）与 Stall 恢复时间** 实验。结果显示 AlphaServe 的总延迟高出 FLEXPIPE 达 66.1%，且在高变异负载下挂起恢复慢了 44% [页码：11, 12]。

2. ServerlessLLM

- 它原本解决什么：[背景知识] 解决 Serverless 大模型服务中的快速实例化问题，通过静态并行性提供低延迟的容器级冷启动和快速资源调配 [页码：11, 14]。
- 它依赖的前提或遗留限制：[基于文中逻辑的推断] 假定并行拓扑是静态配置的，并且物理资源在扩容时不会产生剧烈的 I/O 或网络争抢 [页码：14]。
- 本文批判的具体 limit：[作者明确陈述] 静态并行与资源碎片化的失配，未考虑碎片化物理拓扑和高波动流量的共同影响；当 GPU 空间散落且缺少高速 NVLink 互连时，静态并行极易因跨节点低带宽通信产生巨大阻塞 [页码：14]。
- 本文用哪个模块回应：[基于文中逻辑的推断] **在线流水线重构及自适应流水线弹性伸缩（Adaptive Pipeline Scaling）** 模块，引入基于物理层级资源图（HRG，指将集群中物理服务器、机架和网络连接以树状或图状结构进行层级化拓扑建模的图）的调度以规避硬件冲突 [页码：6, 10]。
- 哪个比较实验检验这一点：[实验支持] **端到端延迟（CV=1, 4）和延迟百分位数** 实验。结果显示在稳定流量下 ServerlessLLM 延迟高出 38.3%，在高变异流量下其吞吐量大幅下降 40.4%，且其 P99 尾部延迟在高变异负载下恶化 2-3 倍 [页码：11, 12, 13]。

3. Tetris

- 它原本解决什么：[背景知识] 解决无服务器推理中的显存限制问题，通过张量共享技术提供高内存效率的多模型 Hosting 方案 [页码：11, 14]。
- 它依赖的前提或遗留限制：[基于文中逻辑的推断] 假定模型可以完全在单个 GPU 上运行或通过张量共享复用，没有考虑流水线并行的多节点扩展 [页码：11, 14]。
- 本文批判的具体 limit：[作者明确陈述] 缺乏流水线并行支持，无法应对大规模模型在严重物理碎片化环境下的分布式执行；在高并发拉起时会产生极大的 I/O 争抢，且实例释放后的缓存失效会导致二次冷启动 [页码：11, 14]。
- 本文用哪个模块回应：[基于文中逻辑的推断] **细粒度模型切分（Fine-Grained Model Partitioning）及自适应流水线弹性伸缩** 模块，引入物理层级控制以及保留宿主主机缓存的带时间衰减亲和度调度 [页码：7, 10]。
- 哪个比较实验检验这一点：[实验支持] **延迟百分位数分布和资源效率测试（CV=4）**。结果显示 Tetris 展现出极高的尾部排队延迟，且消耗 85% 的 GPU 利用率仅达成 1,543 req/s 的吞吐，而 FLEXPIPE 在 43% 利用率下达成 12,000 req/s，资源效率高出 8.5 倍 [页码：12, 13]。

4. MuxServe

- 它原本解决什么：[背景知识] 解决多租户/多模型在单物理 GPU 上的复用效率问题，采用时空统计复用技术最大化物理 GPU 硬件利用率 [页码：11, 14]。
- 它依赖的前提或遗留限制：[基于文中逻辑的推断] 依赖稳定的物理资源可用性假设，认为物理资源不会频繁被强占或重新调配 [页码：14]。

- 本文批判的具体 *limit*：[作者明确陈述] 严重依赖稳定物理资源假设，当物理资源在 Serverless 环境中被频繁抢占和重新调配时表现极差；无法在线重构执行拓扑，在突发洪峰流量下易产生严重的排队拥堵和 SLO 违规 [页码：14]。
- 本文用哪个模块回应：[基于文中逻辑的推断] **在线流水线重构** 模块，引入基于 CV 的多目标得分决策和异步键值缓存（KV Cache，指大模型为避免重复计算而对历史生成词中间结果进行的短期记忆）一致性同步机制 [页码：8, 9]。
- 哪个比较实验检验这一点：[实验支持] **极高波动流量（CV=8）性能测试** 和 **Stall 恢复时间** 实验。结果显示 MuxServe 在流量波动下响应延迟经常飙升并持续超过 10 秒，且在高变异流量下其恢复时间比 FLEXPIPE 慢了 82% [页码：11, 12]。

5. 核心洞见与假设

[基于文中逻辑的推断] **核心洞见与假设**：

1. **以流水线并行应对物理碎片化**：在无法保证高带宽互连的多节点碎片化 GPU 环境下，流水线并行将级间通信转化为点对点的顺序数据传输，通信复杂度由张量并行的 $O(n^2)$ 降为 $O(1)$ ，是解决跨节点大模型协同推理的更佳方式 [页码：3]。
2. **流水线粒度在不同流量下的效益权衡**：细粒度流水线（级数多）单级显存占用低，模型参数加载快（32 级加载耗时仅 5.43s，比 4 级提升 8.7 倍），且支持的最大批处理容量（Max Batch Size）能提升 8 倍（适合流量暴涨时的并发承接），但级间通信开销随级数正比例增加；粗粒度流水线（级数少）级间通信开销极低，但启动慢且支持最大批处理量小 [页码：4, 5] [实验支持]。
3. **深流水线对高变异系数负载的排队吸收**：基于 G/G/S 排队论模型的数学推导，当流量到达变异系数 $CV_a > 3$ 时，由于深流水线能将单级推理服务时间缩短从而消解阶段拥堵，该收益将超过级间累计通信开销的增加。当流水线级数满足 $S \propto \sqrt{CV_a}$ 时可获得最优延迟 [页码：6] [作者明确陈述]。
4. **保留重构潜力的算子级切分与局部一致性维护**：通过在算子级切分时保留 Transformer 架构的注意力块等天然层次结构边界，以及在运行中使用基于有效性掩码（Validity Mask，指用 0 和 1 标记数据是否有效的一组二进制位）的 Token 级异步 KV 缓存同步，可以在不中断服务且决策开销小于 5ms 的前提下，平滑重组流水线拓扑 [页码：7, 8, 9]。

[基于文中逻辑的推断] **落地洞见所需的实现组件**：

1. **Profiler 与图分析器**：获取算子性能指标（计算时间、参数量和激活值大小） [页码：7]。
2. **基于 DP（动态规划）的划分优化器**：求解包含正则项和内存限制的流水线最优切分点 [页码：7]。
3. **在线重构决策器**：实时监测流量特征并计算候选配置得分 [页码：8, 9]。
4. **一致性 KV Cache 同步器**：执行 Token 级掩码点对点异步同步 [页码：9]。
5. **层级资源图（HRG）冲突控制器** 和 **带内存缓存的亲性和性调度器**：规避并发冲突并实现温启动（Warm Start，指利用主机内存中已缓存的模型参数快速初始化实例，而无需从慢速存储重新加载）调度 [页码：10]。
6. **混合数据传输引擎**：RDMA（Remote Direct Memory Access，指允许机器间直接进行内存读取而无需通过双方操作系统的技术）传输与 sendfile 内核态零拷贝传输 [页码：10]。

[基于文中逻辑的推断] **可行性依据与边界**：

- 数学排队论模型推导表明在 $CV_a > 3$ 时，细粒度流水线带来的服务时间缩短能够抵消通信延迟增加 [页码：6] [作者明确陈述]。
- 依赖的前提条件是节点间需支持高带宽、低延迟的 RDMA 物理网络，在纯 TCP/IP 共享网络下，百 GB 级大模型参数在重构时的迁移开销是否会导致服务中断和恢复延迟剧增，尚未得到直接证实 [页码：10]。
- 模型结构假设神经网络具备明显的 Transformer 天然层次结构边界，对于 CNN-Transformer 混合模型、Mamba 状态空间模型等其他非 Transformer 模型，其算子切分正则化约束项是否还能发挥相同作用，文中未加讨论 [页码：7] [背景知识]。

6. 方法：从洞见到可执行系统

[作者明确陈述] **系统整体流程总览**：FLEXPIPE 在离线阶段通过 Profiler 和基于动态规划的 Model Partitioning 优化器，将大模型划分为多粒度的流水线算子方案并存入存储。在线运行中，当用户发送 Prompts 时，API Manager 接收并发送指标到 Adaptive Scaling 进行弹性伸缩决策；同时 Resource Monitor 实时监测运行状态，将 GPU/内存利用率发送给 Scheduler 用于路由更新，将流量 CV 和延迟发送给 Pipeline Refactoring。Pipeline Refactoring 计算当前最佳粒度，触发基于 Token 掩码的 KV Cache 异步同步，在后台不中断推理服务的前提下重构流水线拓扑，最后通过 Scheduler 动态更新路由 [页码：6, 7, 8]。

[基于文中逻辑的推断] **核心模块与层级推理树设计**：

1. 模块 1：Fine-Grained Model Partitioning（细粒度模型切分）

- 动机：解决如何将大模型分解为更小粒度的流水线阶段，并为以后的在线合并与重构保留边界的挑战 [页码：7]。
- 输入与运行步骤：
 - 输入：大模型 M 、计算图 $G = (V, E)$ 、算子性能指标（计算时间 t_c 、参数大小 s_p 、激活值大小 s_a ）以及目标重构周期 C 和显存容量 M_{GPU} [页码：7]。
 - 步骤：采用基于动态规划的算法，在显存上限约束下最小化偏离周期，并通过正则项 $R(S_k)$ 保护 Transformer 块的天然物理边界，同时对齐内存布局 [页码：7]。
 - 输出：包含 K 个阶段的细粒度流水线算子划分方案 $\{S_k\}$ [页码：7]。
- 技术优势：能够保持阶段负载平衡，同时通过保留天然合并点最小化了重构时的重组开销 [页码：7]。
- 与前后模块的连接：离线生成 Executable Model Stages 并存入 Storage，为在线的 Pipeline Refactoring 和 Adaptive Scaling 模块提供支持 [页码：6, 7]。

2. 模块 2：Inflight Pipeline Refactoring（在线流水线重构）

- 动机：解决如何在不断中断服务的情况下自适应切换流水线拓扑以应对流量变异系数波动的挑战 [页码：2, 6]。
 - 输入与运行步骤：
 - 输入：候选配置集 G 、流量变化率（特征速度） ν_t 、当前队列长度 q_j 和变异系数 CV [页码：8, 9]。
 - 步骤：计算多目标决策得分确定最优粒度 g^* ；触发基于 Token 掩码 M_{valid} 的 KV Cache 异步同步，在后台完成转换；转换数据通过 RDMA 或内核态 sendfile 系统调用进行零拷贝传输 [页码：8, 9, 10]。
 - 输出：更新后的流水线拓扑 and 新路由元数据 [页码：9]。
 - 技术优势：重构决策时间小于 5ms，整个拓扑转换过程不挂起推理服务 [页码：9, 10]。
 - 与前后模块的连接：接收来自 Resource Monitor 的 CV 与延迟，决策后向 Performance Controller 发送通知，进一步向 API Manager、Scheduler 和 Adaptive Scaling 广播更新路由信息 [页码：6, 9]。
3. **模块 3：Adaptive Pipeline Scaling（自适应流水线弹性伸缩）**
- 动机：解决如何在碎片化物理资源下实现并发弹性扩展并规避资源争抢与冷启动延迟的挑战 [页码：9, 10]。
 - 输入与运行步骤：
 - 输入：工作负载变异系数 cv_j 、归一化队列长度 $\hat{q}_j$ 、SLO 限时 T_j 、预测的初始化时间 S_j [页码：9, 10]。
 - 步骤：基于 Sigmoid 决策函数确定扩容目标粒度 m_j 并校验 SLO；使用层级资源图（HRG）优化放置路径，避免资源冲突；计算考虑时间衰减因子 $e^{-\lambda(t_{now}-t_s)}$ 的调度亲和力和分数，优先选择有温缓存的主机 [页码：9, 10]。
 - 输出：最适部署的目标服务器 s^* 及扩缩容配置 [页码：10]。
 - 技术优势：物理层拓扑感知避免了大规模扩容冲突，并利用主机保留的参数二级缓存大幅减少了参数重新从慢存储加载的时间 [页码：10]。
 - 与前后模块的连接：接收 API Manager 提供的流量指标，协同 Scheduler 决策部署位置，并根据 Pipeline Refactoring 的反馈优化副本数 [页码：6, 10]。

7. 为什么它可能有效

[基于文中逻辑的推断] **机制与预期收益的逻辑链条：**

1. **机制**：在 DP 划分算法中引入重构正则项，并对齐内存布局 [页码：7]。
 - 预期收益：保留了 Transformer 块天然的物理边界，使在线合并与展开时仅在预定边界处操作，极大降低了显存重新对齐和算子重新切分的开销。
 - 需要的条件：神经网络需为 Transformer 架构，具有可定义的层次化注意力和 MLP 块边界 [页码：7] [背景知识]。
2. **机制**：在线重构基于 Token 级有效性掩码进行 KV Cache 异步同步，并在后台新建实例、合并状态 [页码：9]。
 - 预期收益：在流水线拓扑切换时，推理服务无需中断，重构瞬间不会产生长排队延迟，决策时间小于 5ms，Stall 恢复时间仅 9ms [页码：9, 12]。
 - 需要的条件：节点间需支持高速、低延迟的 RDMA 连接以保证微秒级的网络同步效率 [页码：10]。
3. **机制**：自适应弹性伸缩利用主机内存作为二级缓存（Warm Cache），配合带时间衰减亲和度调度 [页码：10]。
 - 预期收益：避免了参数从慢速持久化存储中重新加载，将冷启动转换为温启动，实例初始化延迟平均缩短了 72% [页码：10, 13]。
 - 需要的条件：宿主机有足够的空闲内存用于缓存释放的 GPU 实例参数 [页码：10]。
4. **机制**：使用物理层级资源图（HRG）来协调服务器、机架和集群资源冲突，优先使用 RDMA/sendfile 传输 [页码：10]。
 - 预期收益：规避了在大规模并发扩容时 PCIe、网络 and 存储 I/O 硬件资源的路径争抢冲突，加快传输速度 [页码：10]。
 - 需要的条件：调度器能准确且实时获取集群的物理拓扑和 I/O 带宽状况 [页码：10]。

8. 实验与指标：证据是否支撑主张

[背景知识] **实验与评估设计**：本实验使用 Microsoft Azure Functions 真实 Serverless 轨迹数据（包括 Top-1 App 和 Top-2 App 等 7 天轨迹），结合 Splitwise 语料库生成 Prompt 文本，基准流量设为 20 QPS 持续 2 小时 [页码：11]。实验所用物理集群由 42 台服务器组成、拥有 82 块 A100 GPU（单 GPU 显存 $\geq 256GB$ ，节点间为 100Gbps 互连网络）的 Kubernetes (v1.23.7) 集群 [页码：10]。对比基线包括 ServerlessLLM、Tetris、AlpaServe、MuxServe 以及相关的吞吐延迟优化与干扰缓解系统 [页码：11]。

比较工作与被批判限制的呼应

比较工作/基线	它试图解决的问题或优势	本文指出的具体 limit/失效条件	本文的对应模块	实际比较实验、指标和结果	仍未验证的边界
AlphaServe [页码：2, 11, 14]	解决模型/流水线并行在 Serving 中的部署；通过历史请求痕迹离线优化流水线结构，最大化长期性能。	缺乏短期动态适应力 ：离线静态配置无法适应 Serverless 工作负载在短时间内的剧烈波动，在高变异流量下会造成严重的排	在线流水线重构 [页码：2, 8]	端到端延迟与吞吐量分解（CV=2, 4）与 Stall 恢复时间。FLEXPIPE 总延迟比其低 66.1%，挂起恢复时间快了 44% [页码：11, 12] [实验支持]。	在超大规模集群中（节点数远大于 42），重构决策算法的计算开销是否依然能控制在 5ms 以内，以及没有低延迟网络时重构表现如何 [页码：9, 10]

		队与流水线空泡 [页码：2, 5]。			[基于文中逻辑的推断]。
ServerlessLLM [页码：11, 14]	解决 Serverless 大模型服务中的快速实例化，通过静态并行性提供低延迟的容器级冷启动和快速资源调配。	静态并行与资源碎片化的失配 ：孤立地优化各组件，未考虑碎片化物理拓扑和高波动流量的共同影响；当 GPU 空间散落且缺少高速 NVLink 互连时，静态并行极易因跨节点低带宽通信产生巨大阻塞 [页码：14]。	在线流水线重构及自适应流水线弹性伸缩 [页码：6, 10]	端到端延迟 (CV=1, 4) 和延迟百分位数。在稳定流量下 ServerlessLLM 延迟高出 38.3%，在高变异流量下其吞吐量下降 40.4% [页码：11, 12, 13] [实验支持]。	在超大规模集群中（节点数远大于 42），层级资源图 (HRG) 的实时冲突路径匹配算法的计算复杂度是否会影响调度速度 [页码：9, 10] [基于文中逻辑的推断]。
Tetris [页码：11, 14]	解决无服务器推理中的显存限制，通过张量共享技术提供高内存效率的多模型 Hosting 方案。	缺乏流水线并行支持 ：无法应对大规模模型在严重物理碎片化环境下的分布式执行；在大规模并发拉起时会产生极大的 I/O 争抢，且实例释放后的缓存失效会导致二次冷启动 [页码：11, 14]。	细粒度模型切分及自适应流水线弹性伸缩 [页码：7, 10]	延迟百分位数分布与资源效率测试 (CV=4)。Tetris 展现出极高的尾部排队延迟，且消耗 85% GPU 利用率仅达成 1,543 req/s 吞吐，而 FLEXPIPE 在 43% 利用率下达成 12,000 req/s，资源效率高出 8.5 倍 [页码：12, 13] [实验支持]。	如果宿主机内存 (Host Memory) 严重受限，二级缓存发生剧烈抖动与换出，温启动调度优势是否会减弱 [页码：10] [基于文中逻辑的推断]。
MuxServe [页码：11, 14]	解决多租户/多模型在单物理 GPU 上的复用效率，采用时空统计复用技术最大化物理 GPU 硬件利用率。	严重依赖稳定物理资源假设 ：当物理资源在 Serverless 环境中被频繁抢占和重新调配时表现极差；无法在线重构执行拓扑，在突发洪峰流量下易产生严重的排队拥堵和 SLO 违规 [页码：14]。	在线流水线重构 [页码：8, 9]	极高波动流量 (CV=8) 性能测试与 Stall 恢复时间。MuxServe 在流量波动下响应延迟经常飙升并持续超过 10 秒，且在高变异流量下其恢复时间比 FLEXPIPE 慢了 82% [页码：11, 12] [实验支持]。	多模型同时在线重构时，相互之间的干扰和资源冲突在更大规模的多租户场景下是否依然可控 [页码：14] [基于文中逻辑的推断]。

主张地图与证据评估

主张	对应实验/表图	直接证据	证据强度	尚未覆盖的条件
主张 1 ：常驻 GPU 预存量由峰值的 75% 降低到 30%，资源分配等待时间缩短 85%，初始化延迟平均缩短了 72% [页码：2, 13]。	文中未提供直观的实时时间线对比图或 CDF 分布图，仅在正文中有文字叙述 [页码：13]。“ 无法从当前 OCR 内容确认 ”图号。	生产集群灰度落地研究的统计数据。常驻长开 GPU 保留配额由历史峰值的 75% 降低到 30% [页码：13] [实验支持]。	中 （由于缺乏对应的可视化直观对比图表和 CDF 分布图支撑）	更大规模集群或不同并发请求曲线下的分配等待时间表现 [页码：13] [基于文中逻辑的推断]。
主张 2 ：在变异系数为 1、2、4 的负载下，总延迟分别比先进系统低 38.3%、46.9% 和 66.1%，且不损害吞吐量 [页码：2, 11]。	端到端延迟与吞吐量分解 (CV=1, 2, 4) 堆叠柱状图 (Figure 8) 以及极高波动 (CV=8) 响应延迟变动折线对比图 (Figure 9) [页码：11, 12]。	在 CV=1 下总延迟低 38.3%；在 CV=2 下低 46.9%；在 CV=4 下比 AlphaServe 低 66.1%，比 MuxServe 低 80.6%，并维持 98.3% 最大可能吞	高 （有详细的端到端时延和吞吐分解量化数据以及图表支持）	非 Transformer 模型架构，或者纯 TCP 网络（无 RDMA 支持）下的变异流量表现 [页码：10] [基于文中逻辑的推断]。

		吐量 [页码：11] [实验支持]。		
主张 3： 在高变异系数 (CV=4) 下，流水线 stall 恢复时间可缩短至 9ms, 比同类系统快 82% [页码：2, 13]。	各系统在不同流量 CV 下的挂起恢复延迟对比柱状图 (Figure 11)，以及重构时序协议说明 (Figure 2(b)) [页码：8, 12]。	在 CV=4 下 FLEXPIPE 的恢复时间缩短至 9ms, 比 AlphaServe 快 44%, 比 MuxServe 和 ServerlessLLM 快 82% [页码：12] [实验支持]。	高 (有中位数恢复时间实验数据和时序机制验证)	超大模型 (如数百 B 参数以上) 在较低带宽网络中的级间 KV Cache 同步与重构时序 [页码：10] [基于文中逻辑的推断]。
主张 4： 在高波动 (CV=4) 下，资源效率比静态系统最高提升达 8.5 倍 [页码：2, 13]。	在不同流量 CV 下各系统的 GPU 利用率与 Goodput 关系图 (Figure 12) [页码：13]。	Tetris 消耗 85% GPU 利用率仅达成 1,543 req/s 吞吐，而 FLEXPIPE 在 43% 利用率下达成 12,000 req/s，资源效率高 8.5 倍 [页码：13] [实验支持]。	高 (有详细的 GPU 利用率与吞吐量的比例对比)	非 Serverless 推理场景 (例如微调、训练等具有静态逆向传播计算图和固定参数同步模式的场景) [页码：10] [基于文中逻辑的推断]。
主张 5： 在生产环境负载和多种模型下，首字预填充 (Prefill) 延迟改进 6.43% 至 24.38%，平均延迟改善 17.32% [页码：2, 13]。	四种模型 (Whisper-9B, Llama-7B, BERT-21B, OPT-66B) 的平均预填充延迟对比柱状图 (Figure 13(a)) 与延迟分布概率密度图 (Figure 13(b)) [页码：13]。	首字预填充延迟改进在 Whisper-9B 改进 6.43%，Llama-7B 改进 19.52%，BERT-21B 改进 18.97%，OPT-66B 改进 24.38%，且尾部延迟分布明显收敛 [页码：13] [实验支持]。	高 (覆盖了多种不同类型的模型与实际生产环境负载)	其他非 Transformer 体系的新型模型架构 (如 Mamba 状态空间模型等) [页码：7] [背景知识]。

9. 图表解读与论证关系

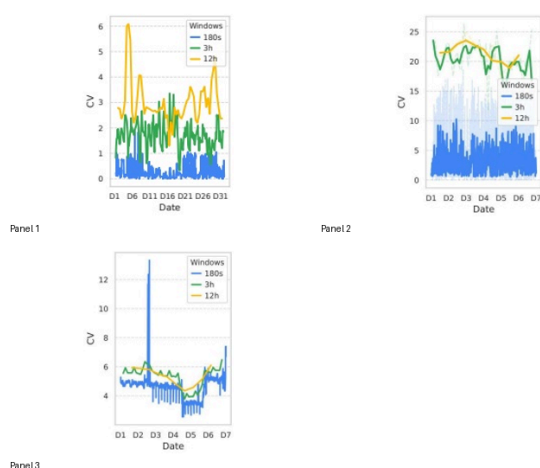


Figure 1. Request distribution CV (coefficient of variation) variations across different periods. Significant mismatches exist in CV calculated with different window sizes (180s, 3h, 12h), $7\times$ variation exists. (a) Request distribution CV of Alibaba Trace, (b) Request distribution CV of Top-1 App, and (c) Top-2 App from Azure [57].

来源：OCR 第 2 页。

图像解读 验证问题：针对 Serverless 大模型 Serving 面临的工作负载波动性 (Workload Volatility) 的根本问题进行检验与量化 [页码：2]。验证真实生产环境的请求到达时间间隔的变异系数 (CV) 在不同滑动窗口 (180s, 3h, 12h) 下存在显著失配，说明请求模式存在高度动态性，无法通过单一静态时间窗口的统计来准确描述，进而证明静态流水线无法适应高变异系数 (CV) 的动态流量变化 [页码：2]。

直接结论：在所有三组 Trace 中，计算得到的 CV 值在不同滑动窗口下存在巨大差异：180s 窗口的 CV 值始终保持在较低水平，而随着滑动窗口增大到 3h 和 12h，计算得出的 CV 普遍显著升高。例如在 Alibaba Trace 中，12h 窗口的 CV 达到了 6 的峰值，而对应的 180s 窗口的 CV 维持在 1 以下，存在约 7 倍的差异；在 Top-1 App 中，12h 窗口的 CV 在 D3 附近达到了接近 24，而 180s 窗口在 5 以下；在 Top-2 App 的 D2.5 附近，180s 窗口表现出高达 13 左右的尖峰，而 12h 窗口在此处相对平缓 (约 6)。

设计含义：该图验证了“真实生产环境的流量 CV 在不同时间窗口下具有极强的波动性与显著失配”这一必要性分析主张 [页码：2]。这直接推动了 FLEXPIPE 的 Inflight Pipeline Refactoring 在线重构决策组件的设计：系统不能依赖离线或大时间窗口的静态划分，必须基于实时监控的流量 CV 和队列长度，在线动态计算候选配置的优化得分，并自适应过渡到最佳流水线阶段粒度 (如高 CV 下采用深度流水线吸收排队，低 CV 下采用粗粒度流水线减少通信开销) [页码：2, 6, 8]。

如何阅读：横轴代表日期 (Date)，Alibaba Trace 从 D1 到 D31，Top-1 App 和 Top-2 App 从 D1 到 D7；纵轴为变异系数 (CV)，Alibaba Trace 范围是 0 到 6，Top-1 App 范围是 0 到 25，Top-2 App 范围是 2 到 13。每个子图中绘制了代表不同滑动窗口大小的折线：180s (蓝色)、3h (绿色)、12h (黄色)。阅读时需比较在相同时间点上，不同时间窗大小计算出的 CV 值差异。

适用范围与证据缺口：本证据覆盖阿里巴巴生产集群 Trace 数据 (31 天) 以及 Microsoft Azure Functions 真实 Serverless 轨迹数据 (包含 Top-1 App 和 Top-2 App 的 7 天数据) [页码：11]。无额外证据缺口。

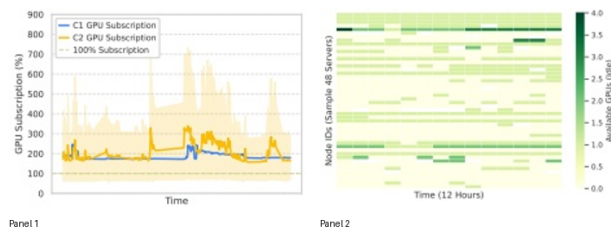


Figure 2. Resource fragmentation in Alibaba. (a) GPU subscription rate averaging 216%, indicating significant resource overcommitment, and (b) Heatmap revealing spatially scattered GPU availability patterns that impede formation of high-bandwidth interconnected GPU groups needed for tensor parallelism. 来源：OCR 第 4 页。

图像解读 验证问题：针对 Serverless 云环境中物理 GPU 资源严重碎片化 (Resource Fragmentation) 与过度承诺的根本冲突进行检验与量化 [页码：1, 3]。旨在验证两个子问题：(1) 生产集群中 GPU 普遍处于严重的共享和过度承诺状态，平均订阅率达 216%，使空闲 GPU 极其稀缺；(2) 可用的 GPU 资源在物理服务器空间上分布极度零散，难以在同一台物理服务器上聚合满足张量并行所需的高带宽互连 GPU 组，从而证明张量并行由于物理拓扑限制在碎片化环境中极易失效 [页码：3, 4]。

直接结论：(a) GPU Subscription Rate 中 C1 和 C2 的 GPU 订阅率在所有时间段都远高于 100% 参考线，主要在 170% 到 340% 之间波动，且 C2 的波动范围上限甚至超过了 700%；(b) GPU Availability Heatmap 中在 12 小时内，绝大多数物理服务器在绝大多数时间内的可用空闲 GPU 数量为 0.0 或低于 1.0 (大面积淡黄色)，只有极少数服务器能在零星的时间段提供 1.0 到 4.0 个空闲 GPU (孤立的绿色斑块)，在时间和空间分布上极度分散和不连续。

设计含义：该图验证了“GPU 资源高超售率以及空闲 GPU 空间极度碎片化分布”的必要性主张 [页码：4]。该事实直接推动了 FLEXPIPE 的流水线并行拓扑选择和层级资源图 (HRG) 分配机制的设计：由于单服务器共定位 4 块 GPU 的概率仅为 0.02%，传统的张量并行被迫降级 [页码：2, 4]，因此 FLEXPIPE 采用点对点通信复杂度低的流水线并行作为分布式 Serving 基础；并设计了层级资源图 (HRG) 来协调服务器、机架和集群资源，采用保留主机内存二级缓存的带时间衰减亲和力和调度策略 [页码：3, 10]。

如何阅读：左侧 (a) GPU Subscription Rate 为折线图，横轴为时间 (Time)，纵轴为 GPU 订阅百分比 (GPU Subscription (%))，范围从 0 到 900%；右侧 (b) GPU Availability Heatmap 为热力图，横轴为时间 (Time (12 Hours))，纵轴为样本物理服务器节点 ID (Node IDs (Sample 48 Servers))，共 48 行，热力图颜色代表可用的空闲 GPU 数量，色条范围为 0.0 (淡黄色) 到 4.0 (深绿色)。

适用范围与证据缺口：本证据覆盖阿里巴巴生产集群 C1 (推理集群，430 节点/468 卡) 与 C2 (混合训练推理集群，927 节点/1175 卡) 的实际 GPU 订阅和可用性统计数据 [页码：4]。无额外证据缺口。

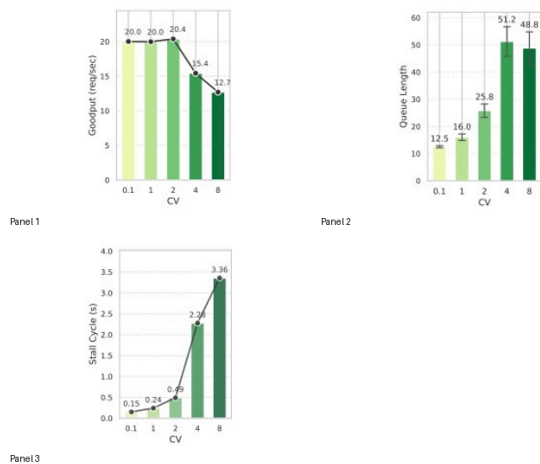


Figure 3. Impact of request distribution variability on pipeline performance. (a) Goodput decreases by 37% as CV increases from 0.1 to 8 due to resource contention; (b) Average queue length grows nearly 4× with increasing CV, indicating pipeline congestion; (c) Stall cycle ratio increases exponentially (22×) at high CV values, showing how static pipelines become inefficient under variable workloads.

来源：OCR 第 5 页。

图像解读 验证问题：量化评估在静态流水线配置下，工作负载波动性对大模型推理流水线性能退化的具体影响，针对“静态流水线与波动负载失配导致排队拥堵和空泡积压”的根问题和旧方法缺陷。旨在验证随着工作负载波动（CV 从 0.1 增加到 8）的增大，系统在吞吐量（Goodput）、平均队列长度（Queue Length）和流水线挂起时间（Stall Cycle）三个维度上的性能恶化情况，说明引入动态调整流水线粒度的必要性 [页码：5]。

直接结论：随着 CV 从 0.1 增加到 8：(a) 吞吐量在 $CV \leq 2$ 时维持在 20.0 至 20.4 req/sec 左右，但在 $CV = 4$ 时下降到 15.4 req/sec，在 $CV = 8$ 时进一步下降到 12.7 req/sec；(b) 平均队列长度从 12.5 逐步上升至 51.2 ($CV = 4$) 和 48.8 ($CV = 8$)；(c) 流水线挂起时间从 0.15s 呈指数级增加至 2.28s ($CV = 4$) 以及 3.36s ($CV = 8$)。

设计含义：该图验证了在静态流水线配置中，高波动负载会引发严重的流水线挂起和队列拥堵，最终导致系统吞吐量大幅退化的主张（吞吐量下降约 37%，队列长度增长近 4 倍，挂起时间增长达 22 倍以上）[页码：5]。这直接启发了 FLEXPIPE 的 Inflight Pipeline Refactoring（在线流水线重构）模块设计：系统不能仅使用固定的粗粒度（如 4 级）流水线，必须在流量波动剧烈时重构为更细粒度的流水线，以将指数级增长的排队等待转化为可控的级间通信开销。但本图仅展示了静态 4 级配置下的性能退化，不能单独证明动态重构或细粒度流水线一定能消除这种退化。

如何阅读：包含三个子图的复合柱状与折线图。横坐标均为工作负载的变异系数（CV），取值分别为 0.1、1、2、4、8。第一个子图纵坐标为吞吐量 Goodput (req/sec)，范围是 0 到 20 以上；第二个子图纵坐标为平均队列长度 Queue Length，范围是 0 到 60；第三个子图纵坐标为挂起时间 Stall Cycle (s)，范围是 0 到 4.0。可通过比较各 CV 值对应的柱子高度和折线点数值来阅读趋势。

适用范围与证据缺口：本实验基于静态 4 阶段 OPT-66B 大模型流水线，在基准流量为 20 QPS 的 2 小时生命周期内，通过调节变异系数 CV (0.1 到 8) 进行测试 [页码：5]。无额外证据缺口。

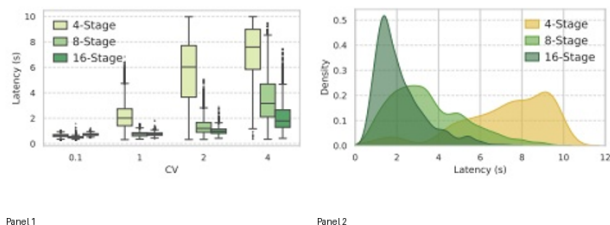


Figure 4. Latency distribution across different request patterns. (a) Box plot comparing pipeline granularities across varying CV values, showing fine-grained pipelines perform better with high-variability workloads; (b) Detailed latency distribution for CV=4 with 4-stage pipeline, revealing significant variance from pipeline stalls.

来源：OCR 第 6 页。

图像解读 验证问题：验证不同流水线阶段数在不同流量变异系数 (CV) 下的性能权衡 (Trade-off) 关系，以此证明在高度波动工作负载下，细粒度 (更多阶段) 的流水线能提供最佳的延迟表现和排队吸收能力。具体验证了 4-Stage、8-Stage 和 16-Stage 三种流水线架构在不同 CV 负载下的延迟分布情况，为系统根据流量动态切换拓扑的决策算法提供机制支持 [页码：6]。

直接结论：(a) Box plot 中，当 CV=0.1 时，16-Stage 的中位数延迟明显高于 4-Stage 和 8-Stage (约 1.2s 对比 0.5s)；随着 CV 增加，所有配置的延迟都有所上升，但 4-Stage 的恶化幅度最大，在 CV=4 时其中位数延迟已升至约 7.6s，而 16-Stage 仍能保持在约 1.8s；(b) Detailed latency distribution 中，在 CV=4 场景下，16-Stage 的延迟分布最窄且最靠左，峰值在 1.4s 附近；8-Stage 的峰值在 3.0s 附近；而 4-Stage 的分布最宽且最靠右，主要在 6s 至 10s 之间，峰值在 9.0s 附近。

设计含义：该图验证了“在低流量变异性下，较粗粒度的流水线因级间通信开销极低而表现出更好的延迟；而在高波动 (CV=4) 下，较细粒度的流水线通过分布式缓冲和更短的单阶段服务时间，能有效吸收并分摊排队延迟，降低平均及尾部延迟”的机制主张 [页码：6]。这一权衡关系直接指出了 FLEXPIPE 在线流水线重构决策模块的依据：没有单一静态配置在所有工作负载下都是最优的，系统必须随着流量 CV 波动而在粗细粒度之间动态切换。但本图仅基于静态配置对比，无法证明重构过程本身的动态开销。

如何阅读：左侧 (a) Box plot 是箱线图，横坐标为 CV (0.1, 1, 2, 4)，纵坐标为 Latency (s) (0 到 10s)，图例标识了 4-Stage (淡黄)、8-Stage (淡绿) 和 16-Stage (深绿) 箱体；右侧 (b) Detailed latency distribution 是概率密度分布图，横坐标为 Latency (s) (0 到 12s)，纵坐标为概率密度 Density (0 到 0.5s 以上)。

适用范围与证据缺口：在恒定总请求量 (基准 20 QPS 持续 2 小时) 和 OPT-66B 模型下，评估 4、8、16 阶段三种静态流水线配置在 CV 范围 0.1 至 4 内的延迟性能 [页码：6]。无额外证据缺口。

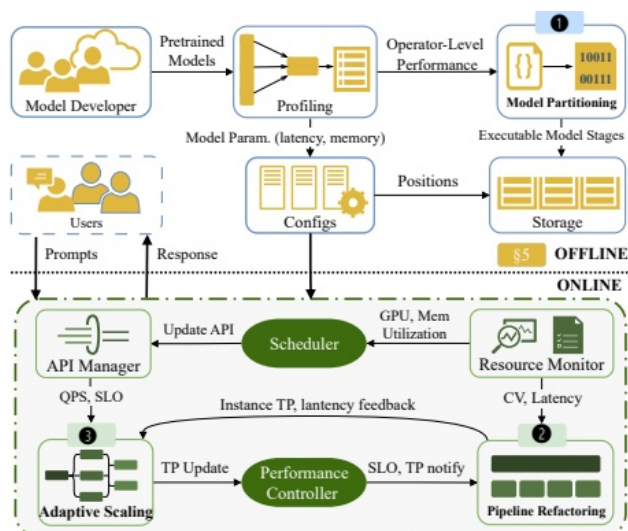


Figure 5. FLEXPIPE system architecture showing the three core components: ① Fine-Grained Pipeline Model Partitioning that decomposes LLMs at operator level for optimal adaptability, ② Inflight Pipeline Refactoring that dynamically adjusts pipeline granularity based on request patterns, and ③ Adaptive Pipeline Scaling that enables efficient resource allocation during traffic fluctuations.

来源：OCR 第 6 页。

图像解读 验证问题：FLEXPIPE 系统的整体架构与 workflow 机制图。旨在说明如何将离线优化与在线适应机制有机结合，以应对 Serverless GPU 推理集群中的流量高波动性与物理资源碎片化难题。试图验证和解释如何通过三个协同组件 (① 离线算子级模型切分、② 在线流水线重构、③ 自适应弹性伸缩) 以及系统各个角色的交互来实现不中断推理服务的动态流水线重构控制回路 [页码：6]。

直接结论：(1) 系统明确解耦为离线静态多粒度切分 (OFFLINE) 和在线动态重组自适应 (ONLINE) 两个处理阶段；(2) 三个核心创新模块分别被标为带黑圈数字：① Model Partitioning (模型切分)、② Pipeline Refactoring (流水线重构)、③ Adaptive Scaling (自适应弹性伸缩)；(3) Resource Monitor 解耦了控制回路：将硬件资源利用率发送至 Scheduler 驱动实例路由，将工作负载变异特征 (CV) 和延迟发送至 Pipeline Refactoring 触发拓扑重构；(4) 图中存在一处拼写错误：在 Pipeline Refactoring 指向 Adaptive Scaling 的反馈箭头中，latency 被错写为 "latency"。

设计含义：该图验证了系统控制回路与组件解耦机制。该设计通过离线多粒度算子切分 (①) 为运行时的快速重构提供了参数边界和布局 Configs；在线运行中，Resource Monitor 通过实时流量的变异系数 (CV) 和延迟输入给 Pipeline Refactoring (②)，当流量波动 (CV 增加) 时触发拓扑重构 (如切分为更细粒度阶段)，或者在流量稳定时合并为粗粒度阶段以降低通信开销；同时结合 Adaptive Scaling (③) 基于 QPS 和 SLO 实现实例级的弹性扩缩容，从而在宏观上协同解决了 GPU 资源碎片化和流量剧烈波动的双重挑战 [页码：6, 7, 8]。然而，作为系统概念机制图，该图本身无法用以定量证明 FLEXPIPE 实际相比于 AlphaServe、ServerlessLLM 等基线系统在运行中的 P99 延迟降低幅度、Stall 恢复时间或资源效率的具体提升倍数。

如何阅读：该图以水平黑色虚线为界分为上下两部分。上半部分为离线阶段 (§5 OFFLINE, yellow 圆角框)，包含 "Model Developer"、"Profiling"、"Configs"、"Model Partitioning" (①) 以及 "Storage" 模块；下半部分为在线运行阶段 (绿色点划线框包围, ONLINE)，展示了 Users 以及 "API Manager"、"Scheduler"、"Resource Monitor"、"Adaptive Scaling" (③)、"Pipeline Refactoring" (②) 和 "Performance Controller"。阅读顺序从离线到在线、从左到右依次进行。

适用范围与证据缺口：本系统架构与 workflow 适用于托管在 Kubernetes Serverless GPU 集群上、需要多节点流水线并行分布式协作的大语言模型推理服务场景 [页码：6]。无额外证据缺口。

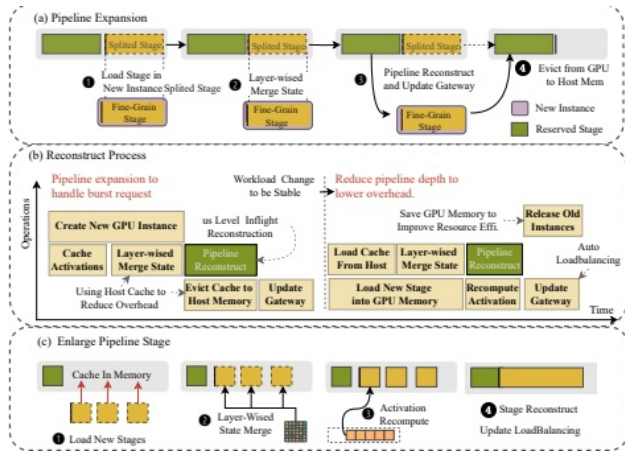


Figure 6. Inflight pipeline refactoring mechanism. (a) Stage refinement process where fine-grained partitioning occurs by evicting parameters and redistributing them onto additional GPUs; (b) Temporal sequence diagram showing synchronization protocol during refactoring; (c) Stage consolidation process where parameters from multiple stages are merged, utilizing host memory caching to minimize loading overhead from per-sistent storage.

来源：OCR 第 8 页。

图像解读 验证问题：FLEXPIPE 在线流水线重构与同步协议的机制示意图。旨在解释如何在运行时（Inflight）不挂起或不中断推理服务的前提下，实现流水线级数的展开（Expansion）与合并（Enlarge）。该图具体展示了：流水线展开时的参数分配与迁移操作 (a)、重构操作在时间轴（Time）上的重叠与异步同步时序协议 (b)、以及流水线合并时的内存缓存利用与状态合并操作 (c) [页码：8]。

直接结论：(1) 流水线展开过程 (a) 由四个核心步骤 (● 加载新实例、● 状态合并、● 重构和网关更新、● 驱逐到 Host Memory) 组成；(2) 重构协议在 (b) 中实现了多项操作的并行与重叠，如在展开时，“Create New GPU Instance”与“Cache Activations”和“Layer-wised Merge State”存在时间上的重合，重构的同时进行“Evict Cache to Host Memory”和“Update Gateway”；(3) 合并过程 (c) 依赖“Cache In Memory”机制，首先通过 ● “Load New Stages”重用 Host 内存中的 cached 阶段参数，再经过 ● “Layer-Wised State Merge”、● “Activation Recompute”并最终在 ● 完成“Stage Reconstruct”并更新负载均衡；(4) 重构协议图 (b) 明确标识其重构决策与同步发生是在微秒级 (“us Level Inflight Reconstruction”)；(5) 图片中包含多处英文拼写与语法小瑕疵：如 (a) 中的“Splited Stage”和“Layer-wised”，(b) 中的“us Level”与“Resource Effi.”，以及 (c) 中的“Layer-Wised”和“LoadBalancing”。

设计含义：该图验证了在线不中断重构与缓存局部性机制。该设计通过 (b) 中时序的重合性（如在新建 GPU 实例的同时在后台进行 KV Cache 的异步同步和 Layer-wised 状态合并），证明了重构决策与切换可以在不中断前端推理服务（即小于 5ms 决策，9ms Stall 恢复）的前提下平滑完成 [页码：8, 9, 12]。(a) 与 (c) 中的“Evict from GPU to Host Mem”与“Cache In Memory”机制展示了如何利用 Host Memory 作为参数的二级缓存（Warm Cache），从而避免了在流水线级数合并或重新调整时再次从慢速持久化存储中读取参数的巨大冷启动开销，这直接支持了公式 13 中关于亲和力和调度缓存温启动的设计合理性 [页码：10]。但该图仅为定性机制图，无法用于定量验证在具体基线对比中在特定 CV 流量下的吞吐量差异和具体硬件上 RDMA 与 TCP 网络下的传输时延数值。

如何阅读：从上到下分为三个面板。(a) Pipeline Expansion：从左向右按步骤 ● ● ● ● 顺序阅读以了解设备间参数分发；(b) Reconstruct Process：沿着 Time 轴从左向右对比左侧（展开时后台并行进行状态合并与 μs 级不中断重构）与右侧（合并时从 Host 恢复 Cache）；(c) Enlarge Pipeline Stage：从左向右按步骤 ● ● ● ● 顺序阅读以了解 Host 内存缓存重用与状态合并。

适用范围与证据缺口：适用于大语言模型 (LLMs) 在运行时 (Inflight) 动态改变流水线级数（如 4 级与 32 级之间转换）以及进行 KV 缓存异步一致性同步的分布式推理场景 [页码：8]。存在 OCR/图题冲突：OCR 候选图注中描述 (a) 为“Stage refinement process”，(c) 为“Stage consolidation process”；然而在实际图片中，(a) 的标题为“(a) Pipeline Expansion”，(c) 的标题为“(c) Enlarge Pipeline Stage”，存在文字表述的不一致 [页码：8]。

10. 完整因果推理树

- **根问题：**[作者明确陈述] Serverless 集群中，高度波动的流量（CV 波动达 7 倍）与碎片化的 GPU 资源（同机共定位概率仅 0.02%）使得传统紧密互连的张量并行失效，使用静态配置会导致资源效率低下（仅 17% 利用率）且极易引起 SLO 违规 [页码：1, 2, 3]。
- **旧方法限制 1：**[作者明确陈述] 依赖静态流水线配置（如 AlphaServe），离线参数分区无法在运行时动态重组，在高波动流量下会导致严重的排队拥堵和流水线空泡 [页码：2, 5]。
 - **核心洞见/假设 1：**[作者明确陈述] 流水线级数存在启动时间（细粒度快）与级间通信延迟（粗粒度少）的效益折衷，在流量波动剧烈（ $CV > 3$ ）时，切分为更深流水线（ $S \propto \sqrt{CV_d}$ ）可以通过缩短单级推理服务时间来有效吸收排队延迟 [页码：4, 5, 6]。
- **方法模块 1 & 2：**[基于文中逻辑的推断] **细粒度模型切分 (Fine-Grained Model Partitioning)** 与 **在线流水线重构 (Inflight Pipeline Refactoring)**：离线采用带正则项的 DP 算法切分，在线监控流量 CV 并计算多目标决策得分，通过基于 Token 掩码的 KV Cache 异步同步在后台微秒级完成拓扑切换 [页码：7, 8, 9]。

- ▶ **可验证预测 1.1**：[作者明确陈述] 在变异系数为 1、2、4 的负载下，总延迟分别比先进系统低 38.3%、46.9% 和 66.1%，且仍能维持最大可能吞吐量的 98.3% [页码：2, 11]。
 - **指标与实验 1.1**：[实验支持] 端到端延迟与吞吐量分解实验，测量在不同 CV 负载下的总延迟、排队时间、级间通信时间以及最大吞吐量 (Goodput) [页码：11]。
 - **图表证据 1.1**：[实验支持] 不同 CV 组别下的延迟分布箱线图 (第 6 页图片 2 Panel 1) 和 CV=4 下延迟概率密度图 (第 6 页图片 2 Panel 2) [页码：6]；端到端延迟细分的堆叠柱状图 (文中提及 Figure 8) [页码：11]。
 - ▶ **结论与适用边界 1.1**：[实验支持] 细粒度流水线能够有效在流量暴涨时吸收排队延迟，但该结论仅适用于具备清晰 Transformer 天然层次结构边界的模型 [页码：7, 11]。
- ▶ **可验证预测 1.2**：[作者明确陈述] 在高变异系数 (CV=4) 下，流水线 stall 恢复时间可缩短至 9ms，比同类系统快 82% [页码：2, 13]。
 - **指标与实验 1.2**：[实验支持] 流水线挂起恢复时间实验，测定中位数恢复时间 [页码：12]。
 - **图表证据 1.2**：[实验支持] 各系统在不同流量 CV 下的挂起恢复延迟对比柱状图 (Figure 11) 和在线不中断重构时序协议图 (第 8 页图片 2 Panel (b)) [页码：8, 12]。
 - ▶ **结论与适用边界 1.2**：[实验支持] 基于 Token 级掩码的异步 KV 缓存同步能实现 9ms 的极速恢复，但其极度依赖高带宽、低延迟的 RDMA 物理网络，在纯 TCP/IP 共享网络中尚未被直接证实能达到此效果 [页码：10, 12]。
- ▶ **旧方法限制 2**：[作者明确陈述] 现有的弹性伸缩系统 (如 ServerlessLLM 和 Tetris) 专注于冷启动优化或张量共享，但缺乏对执行拓扑的动态在线调整能力，且在大规模并发扩容时会面临 PCIe 和网络带宽争抢 [页码：11, 14]。
 - **核心洞见/假设 2**：[基于文中逻辑的推断] 通过层级资源图 (HRG) 协调物理争抢，结合主机内存二级缓存与时间衰减亲和度调度，可以将大模型冷启动转化为温启动并避免 I/O 冲突 [页码：10]。
 - **方法模块 3**：[基于文中逻辑的推断] **自适应流水线弹性伸缩 (Adaptive Pipeline Scaling)**：利用 HRG 冲突控制器优化放置路径，计算考虑时间衰减因子 $e^{-\lambda(t_{now}-t_s)}$ 的服务器亲和度得分 [页码：10]。
- ▶ **可验证预测 2.1**：[作者明确陈述] 在生产环境的 Serverless 集群中灰度落地，常驻长开 GPU 保留配额由峰值 75% 降低到 30%，资源分配等待时间缩短 85%，实例初始化延迟平均缩短 72% [页码：2, 13]。
 - **指标与实验 2.1**：[实验支持] 真实生产集群灰度落地研究，监测分配等待时间、初始化延迟和长开 GPU 保留量 [页码：13, 14]。
 - **图表证据 2.1**：[基于文中逻辑的推断] 分配的物理 GPU 数量随流量波动的实时时间线对比图、资源分配等待时间和初始化延迟的 CDF 图 (文中未说明图号，未提供可视化图表，仅文字陈述) [页码：13]。**“无法从当前 OCR 内容确认”**其图表视觉设计。
 - ▶ **结论与适用边界 2.1**：[实验支持] 温缓存亲和度调度能显著降低预留资源和 cold start 延迟，但其适用边界仅限于宿主主机内存充足、温缓存未被剧烈驱逐的场景 [页码：10, 13]。
- ▶ **可验证预测 2.2**：[作者明确陈述] 在高变异负载 (CV=4) 下，资源效率比静态系统 (如 Tetris) 最高提升达 8.5 倍 [页码：2, 13]。
 - **指标与实验 2.2**：[实验支持] 资源利用效率实验，分析不同 CV 下吞吐量与实际消耗 GPU 利用率的关系 [页码：13]。
 - **图表证据 2.2**：[实验支持] 各系统的 GPU 利用率与 Goodput 关系图 (Figure 12) [页码：13]。
 - ▶ **结论与适用边界 2.2**：[实验支持] 物理层拓扑感知和自适应伸缩能将无效的级间冲突开销降至最低，大幅提升高并发下的资源利用率，但此结论在非 Serverless 推理场景 (如大模型持续训练) 下不适用 [页码：10, 13]。

11. 结论、贡献与边界

[作者明确陈述] **贡献总结**：

1. 提出一种在运行中动态响应请求分布变化、无需中断服务而调整流水线拓扑的自适应大模型推理 Serving 方法 [页码：2]。
2. 设计了一种结合算子级开销剖析与重构潜力约束的细粒度大模型切分方法 [页码：2]。
3. 提出一种基于物理资源层级协调 (层级资源图 HRG) 与亲和度调度的 Serverless 大模型弹性伸缩机制 [页码：2]。

[实验支持] **证据充分的结论**：

- 在高度波动负载 (CV=4) 下，FLEXPIPE 的端到端总延迟显著降低 (比 AlphaServe 低 66.1%，比 MuxServe 低 80.6%)，吞吐量几乎无损 (维持最大可能吞吐的 98.3%) [页码：11]。
- 在高度波动负载下实现了 9ms 的极速挂起恢复，比 MuxServe 和 ServerlessLLM 快 82% [页码：12]。
- 真实生产集群灰度落地结果表明，资源分配等待时间缩短 85%，实例初始化延迟平均缩短 72%，常驻 GPU 保留配额由峰值的 75% 降低到 30% [页码：13, 14]。
- 在高度波动流量下，资源效率相比 Tetris 最高提升达 8.5 倍 [页码：13]。

[基于文中逻辑 of 推断] **尚属推断的意义**：

- 该方法在运行时进行无缝的流水线拓扑调整，揭示了将“排队延迟”主动转化为“通信开销”在吸收高变异负载时的数学与系统级优势，为未来更大规模的 Serverless 异构计算资源管理提供了新范式。

[基于文中逻辑 of 推断] **适用条件**：

- 适用于工作负载波动性极高 (CV ≥ 3) 且物理 GPU 资源存在严重空间碎片化的在线 Serverless LLM 推理 Serving 场景 [页码：1, 14]。
- 适用于网络支持高带宽、低延迟点对点传输 (如 RDMA 网络) 的环境 [页码：10]。
- 适用于具有天然 Transformer 架构逻辑边界的神经网络模型 [页码：7] [背景知识]。

[基于文中逻辑 of 推断] **局限与下一步验证**：

- **网络限制**：在普通的 TCP/IP 共享网络下，百 GB 级模型参数在重构时的迁移开销是否会导致服务中断和恢复延迟剧增，需要进行混合网络环境下的下一步验证 [页码：10]。
- **架构限制**：该算法如何推广到非 Transformer 架构的模型（如 CNN-Transformer 混合模型、Mamba 状态空间模型等），需要下一步探索 [页码：7] [背景知识]。
- **集群规模限制**：在成百上千节点的超大规模集群中，层级资源图（HRG）的构建及冲突路径实时匹配算法的计算复杂度是否能保持在 5ms 以内，需要进一步验证 [页码：9, 10]。

[基于文中逻辑 of 推断] **审稿式风险检查**：

- **贡献新意**：当前证据未显示该风险。该论文首次将 Inflight Pipeline Refactoring 引入 Serverless 大模型 Serving，并通过排队论推导提供了坚实的理论支撑，新意明显 [页码：2, 6]。
- **写作/可复现性**：存在中度风险。关键公式中控制参数的整定依据与取值完全缺失（如公式 4 中的压缩因子 α 、公式 10 中的 γ_0 和 α 、公式 11 中的 β 和 γ 、公式 13 中的衰减常数 λ 均“无法从当前 OCR 内容确认”），且开源代码库链接缺失（“文中未说明”），给独立复现带来较大困难 [页码：7, 8, 9, 10]。
- **效果大小**：当前证据未显示该风险。实验表明在高度波动下延迟降低达 66.1%-80.6%，资源利用效率提升 8.5 倍，常驻 GPU 保留率由 75% 降至 30%，效果显著 [页码：11, 13]。
- **实验覆盖**：存在轻度风险。实验局限于中等规模的集群（42 台服务器，82 块 GPU） [页码：10]，且有 7 张图表未进行视觉分析，这使得系统在更大规模集群中的通信与调度开销未得到直接量化印证 [页码：10]。
- **方法设计**：存在中度风险。系统极其依赖低延迟的 RDMA 物理网络来实现微秒级的在线重构同步，但在没有 RDMA 支持的 TCP 共享网络下的实际表现和衰减情况缺乏直接的实验数据对比，这限制了方法在普通云环境中的普适性 [页码：10]。

12. 面向初学者的学习路径

1. **无服务器架构（Serverless）**：[背景知识] 指按实际计算量付费且免去人工服务器运维的云计算模式。理解它能帮我们明白为什么系统要应对剧烈波动的流量，以及为什么要极力降低常驻 GPU 预留配额以节省成本。
2. **流水线并行（Pipeline Parallelism）**：[背景知识] 指将模型切分成多个连续阶段，让不同 GPU 像流水线工人一样依次处理部分层的分布式计算方式。这是理解 FLEXPIPE 动态调整“流水线并行粒度”这一核心机制的基础。
3. **工作负载波动性与变异系数（CV）**：[背景知识] 衡量流量不稳定程度的指标，值越大代表流量忽大忽小越剧烈。这是理解为什么静态配置会失效、以及为什么系统需要根据 CV 在粗细粒度拓扑之间自适应切换的前提。
4. **键值缓存（KV Cache）**：[背景知识] 指大模型推理时为避免重复计算而对历史生成词中间结果进行的短期记忆。这是理解为什么在线重构必须在后台进行“KV Cache 异步同步”以保证推理服务不中断的关键。